MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

The Agent Loop & Design Patterns

How per-request model calls become a stateful agent: one ReAct-inspired loop, a runtime that owns the state, and the four patterns that organise everything we build this semester.

Unit 02 · Session 02 · Week 2
Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Where we are** from classical architectures to executable mechanics
2. **Anatomy of the loop** reason, act, observe, decide — and where state lives
3. **A minimal agent in ~50 lines** the poster code, dissected part by part
4. **Stop conditions & failure modes** infinite loops, context overflow, derailment
5. **Token economics** why unbounded full-history loops grow quadratically
6. **Four design patterns** Planning, Tool Use, Reflection, Multi-Agent — the semester roadmap

Where we are

Session 01 gave us definitions and history; today we open the machine.

Last week — agent = LLM + tools + loop + goal; workflows vs. agents; perceive-reason-act, subsumption, BDI as precursors.

What survived — the sense-think-act cycle — every classical architecture iterates perception and action.

Today — that cycle as running code: the loop, its state, its stop conditions, its costs.

The lens — no frameworks yet — plain Python against a raw model API, so nothing is hidden.

Lab 02 — you build this loop around ollama.chat and a small offline mini-web, then break it on purpose.

ONE IDEA TODAY

In our minimal implementation, an agent is a loop around model requests, each conditioned on the context we supply. Memory, planning and safety are elaborations of that loop.

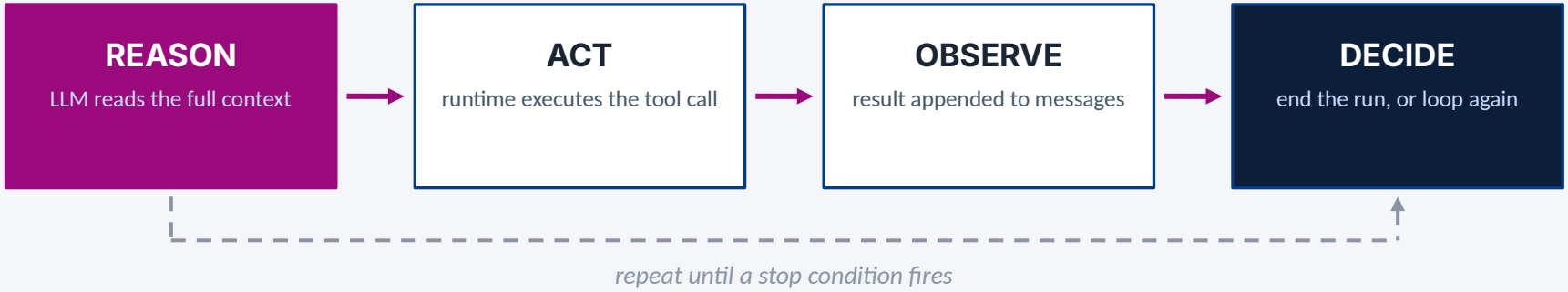
1

Anatomy of the agent loop

Four phases, one growing message list, three kinds of state

The loop: reason, act, observe, decide

One iteration = one LLM call, possibly tool executions — a ReAct-inspired loop (cf. Yao et al., 2023).



- The model never executes anything — it only emits text or a structured tool call.
- A tool call means “loop again”. No tool call ends the turn — but classify it: completed, refused, truncated or failed.

One iteration under the microscope

Who does what in a single pass through the loop

Reason — the message list is sent to the model; nothing else is "in its head" for this request.

Act — your runtime — plain Python — parses the tool call and executes it, with real privileges.

Observe — the tool result becomes a message; the model only sees it on the next call.

Decide — no explicit branch in the model — the output type itself is the control signal.

Consequence — capabilities and risks live in the runtime, not the model.

DIVISION OF LABOUR

The model proposes; the runtime disposes. Tool execution, permissions and budgets are your code — and your responsibility.

State: model, runtime, environment

The context is the model-visible working state; runtime and environment state exist too.

Model (per request)

Each request is conditioned on the supplied context; weights stay fixed during inference. Sampling randomness and provider-side session state are the exceptions to “stateless”.

Runtime (the agent)

Your code owns the message list, step and token counters, pending tool calls, permissions and stop logic. Resuming a run means restoring all of it, not just messages.

Environment (the world)

Files written, pages fetched, messages sent, external systems changed. Side effects persist whether or not they are reflected in the context.

2

A minimal agent in ~50 lines

The poster code of this course, dissected — cloud-API version; Lab 02 ports it to Ollama

Part 1 of 4 — one tool: schema and dispatch table

Cloud example (OpenAI SDK). The model never sees the Python function — only the JSON description.

mini_agent.py (1/4)

```
import json, time
from openai import OpenAI   # cloud example; Lab 02: ollama

client = OpenAI()           # or base_url of a local server
MODEL = "gpt-5-mini"        # any tool-capable model
def web_search(query: str) -> str:
    """Search the web, return top hits as text."""
    ...  # call a real search API here

TOOLS = [{"type": "function", "function": {
    "name": "web_search",
    "description": "Search the web for a query.",
    "parameters": {"type": "object",
        "properties": {"query": {"type": "string"}},
        "required": ["query"]}}}]
FUNCS = {"web_search": web_search}   # name -> callable
```

Part 2 of 4 — calling the model, with bounded retries

API and network errors are handled outside the loop's step count — and never forever.

mini_agent.py (2/4)

```
def call_model(messages, retries=3):
    """One model request; bounded retries on API errors."""
    for attempt in range(retries):
        try:
            return client.chat.completions.create(
                model=MODEL, messages=messages, tools=TOOLS)
        except Exception as e:          # API or network error
            print("API error:", e)
            time.sleep(2 ** attempt)    # back off, then retry
    raise RuntimeError("model unavailable after retries")

messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": task},
]
```

Part 3 of 4 — act: execute one requested call safely

Dispatch by the requested name; invalid JSON, unknown tool and bad arguments become observations.

mini_agent.py (3/4)

```
def run_tool(call):
    """Run one requested call; errors become observations."""
    name = call.function.name
    try:
        fn = FUNCS[name]            # unknown tool -> KeyError
        args = json.loads(call.function.arguments)
        result = fn(**args)         # missing arg -> TypeError
    except KeyError:
        result = f"TOOL ERROR: unknown tool {name!r}"
    except (json.JSONDecodeError, TypeError) as e:
        result = f"TOOL ERROR: bad arguments: {e}"
    except Exception as e:
        result = f"TOOL ERROR: {e}"
    return {"role": "tool", "tool_call_id": call.id,
            "content": str(result)[:4000]}  # chars, not tok.
```

Part 4 of 4 — the loop: reason, observe, exit

One request per iteration; the context grows by appending; the exit is classified, not assumed.

mini_agent.py (4/4)

```
for step in range(MAX_STEPS):           # stop: step cap
    resp = call_model(messages)         # REASON
    msg = resp.choices[0].message
    messages.append(msg)                # state grows here

    if not msg.tool_calls:              # no action requested
        status = resp.choices[0].finish_reason
        if status == "length":          # truncated, not done
            print("Answer cut off at token limit."); break
        print(msg.content); break   # completed (or refused)

    for call in msg.tool_calls:         # ACT, one per call
        messages.append(run_tool(call)) # OBSERVE
else:
    print("Step budget exhausted, no answer.")
```

Fifty lines, six design decisions

Every agent framework is an elaboration of choices already visible here.

Tool schema — the model knows only what the JSON says — interface design is prompt design.

Step cap — MAX_STEPS caps iterations only — not tool calls per turn, wall-clock time or tokens per call.

Error policy — invalid JSON, unknown tool, missing argument: each becomes a tool message with its tool_call_id.

Bounded retries — API failures are retried a few times with back-off — then the run fails loudly, not silently.

Truncation — [:4000] counts characters, not tokens; it bounds one observation, not the whole request.

Termination — no tool call ends the turn, but check finish_reason: completed, refused, truncated or failed.

COMPARE WITH LAB 02

The lab builds the same loop around ollama.chat with a local qwen2.5:7b. Arguments arrive as a dict and results carry tool_name — but every guard shown here is still needed.

3

Stop conditions & failure modes

How loops end — and how they go wrong

Four ways a loop should end

Production agents implement all four and classify how the run ended; the minimal agent has only the first two.

No further action

The model emits no tool call. Not automatically success: check finish_reason — completed, refused, truncated and empty answers look alike at first.

Step cap

Hard iteration limit (MAX_STEPS). Blunt but non-negotiable: the last defence when the model never converges. Limits iterations, not work per iteration.

Budget cap

Stop on cumulative tokens, money or wall-clock time — budgets a step cap cannot express. One step can be arbitrarily long.

Human interrupt

Approval gates before irreversible actions, plus a kill switch. Anthropic's guidance: add checkpoints where errors are costly.

Three common failure modes

No exception is raised in any of them — the pathology lives at trajectory level. Guardrail engineering follows in Unit 7.

Failure mode	What happens	Early symptom	First-line defence
Infinite loop	same call, same arguments, hoping for a different result	identical tool call repeated across steps	step cap; duplicate-call detector
Context overflow	messages outgrow the window; requests fail or old context drops	latency and cost rise every step	truncate observations; summarise old turns
Derailment	locally reasonable steps drift away from the stated goal	actions stop serving the goal	re-inject goal; plan checkpoints; human review

These arise from interactions between model, context, tools and control logic — rarely from a single cause.

Case study (synthetic): a web-research run derails

Constructed trace. Task: "Write a sourced report on the EU AI Act's rules for general-purpose models."

Steps 1–3 — sensible searches, two good primary sources retrieved and summarised.

Step 4 — one source discusses US executive orders; the agent treats this as "relevant context".

Steps 5–9 — it now researches US policy — each step locally reasonable, globally off-task.

Plausible factor — the goal is one line, 20k tokens back; recent observations dominate — a hypothesis, not a proven root cause.

Test it — ablate: same material with a goal reminder, a shorter context, a changed tool result; compare trajectories.

Fixes — restate the goal every iteration, add a plan to check off, cap steps per subtopic.

KEY INSIGHT

Derailment is rarely one bad step. Model behaviour, context layout, tool results and control logic compound — so prompting alone seldom removes it. Lab 02, Experiment 4 reproduces it with a bait page.

4

Token economics of loops

Why full-history loops grow quadratically — and what to do about it

The quadratic tax

Cumulative input volume in an unbounded full-history loop grows quadratically with steps.

Each iteration resends the whole history. With $I_k = I_0 + (k-1)\Delta$ input tokens at step k, the sum over n steps is $n \cdot I_0 + \Delta \cdot n(n-1)/2$. For $I_0 = \Delta = 2{,}000$ and $n = 20$: 420,000 cumulative input tokens — about 10x the final context of 40,000. Input tokens, not euros.

Managing the cost of a loop

Economics, not luxuries: these decide whether an agent is deployable.

Prompt caching — providers reuse the unchanged prefix; the discount on cached input depends on provider, model and tariff.

Observation hygiene — truncate, filter and summarise tool results before they enter the context.

Compaction — periodically replace old turns with a summary — trades fidelity for headroom.

Model routing — cheap model for mechanical steps, expensive model for planning (more in Session 05).

Budget caps — the stop condition and the cost model are the same mechanism seen twice.

RULE OF THUMB

Input volume is $n \cdot I_0 + \Delta \cdot n(n-1)/2$ tokens. Price separately: uncached input, cache reads and writes, output including billable reasoning, and tool costs. Shorten observations before shortening the loop.

5

Four agentic design patterns

Ng's taxonomy as our semester roadmap

The four patterns (Ng, 2024)

Each is a disciplined way to spend extra tokens for better behaviour.

Planning

Decompose the goal into explicit subgoals before acting; execute and replan against the plan. Counters derailment. Sessions 05-06.

Tool Use

Extend the model with external actions: search, code execution, APIs. The pattern our minimal agent already implements. Sessions 03 and 10.

Reflection

The agent critiques and revises its own output — draft, review, redraft. Cheap quality gains, but models grade their own homework. Session 12.

Multi-Agent

Specialised roles (planner, researcher, critic) with separate contexts collaborate. Powerful for breadth; costly and failure-prone. Session 12.

When each pattern pays off

Patterns are token-for-quality trades — apply them where the trade is favourable.

Pattern	Pays off when	Main cost	Watch out for
Planning	long tasks with checkable milestones	upfront tokens, rigidity	plans that ignore new evidence
Tool Use	facts or actions beyond the model	engineering, security surface	poor tool descriptions mislead model
Reflection	quality gaps the model can spot	2-3x tokens per output	self-praise instead of critique
Multi-Agent	wide tasks split into roles	context loss between agents	cost explosion, telephone game

The labs add these patterns one at a time over the semester; Lab 02 already has Tool Use.

Lab 02 this week: build the loop, then break it

Gap-fill notebook, local Ollama model (qwen2.5:7b), offline mini-web in data/ — no API keys, 90-120 min.

Parts B-C — two tools (web_search, read_page) over seven synthetic pages; the minimal loop around ollama.chat.

Part D — instrument each iteration: context size, prompt and output tokens, latency — see the quadratic growth and caching.

Part E — four break-it experiments: runaway loop, step cap as backstop, oversized pages, derailment bait.

Part F — first-line defences: budget cap, duplicate-call detector, goal re-injection.

Report R1-R5 — stop-condition families, your own quadratic-tax estimate, two defences with trajectories, next pattern.

Baseline set — keep a small fixed task set with tokens, latency and success from now on — reused until Session 14.

BREAK IT ON PURPOSE

Feed the loop invalid JSON arguments, an unknown tool name and a call with a missing required argument. Each must become a tool message the model can read — never a crash.

TAKEAWAYS

What you should remember

- **The loop is the agent —** reason, act, observe, decide — a ReAct-inspired loop around per-request model calls.
- **Context is working state —** the model-visible context; runtime counters, tool status and environment state live outside it.
- **Stops are designed —** no further action, step cap, budget cap, human interrupt — and classify how the run ended.
- **Failures interact —** infinite loops, context overflow and derailment arise from model, context, tools and control logic together.
- **Input grows quadratically —** in an unbounded full-history loop; caching, truncation and compaction are necessities, not optimisations.
- **Four patterns ahead —** Planning, Tool Use, Reflection, Multi-Agent structure the rest of the semester.

NEXT SESSION Tools — function calling mechanics and the Model Context Protocol.

Reading & references

- Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.
  The canonical reason-act-observe formulation; read sections 1-3.
- Anthropic (2024). Building Effective Agents. anthropic.com/engineering/building-effective-agents.
  Workflow vs. agent distinction, stop conditions, "start simple" guidance.
- Ng, A. (2024). Agentic Design Patterns, parts 1-5. The Batch, DeepLearning.AI.
  Source of the four-pattern taxonomy used as our roadmap.
- Wang, L. et al. (2024). A Survey on Large Language Model Based Autonomous Agents. Frontiers of Computer Science 18(6).
  Broad survey; useful for the loop-and-memory framing.
- Shinn, N. et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. arXiv:2303.11366.
  Preview of the Reflection pattern, treated in depth in Session 12.
- Schick, T. et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. arXiv:2302.04761.
  Background for the Tool Use pattern before Session 03.